אוניברסיטת בן-גוריון
מדור בחינות

מספר נבחן:_____

תאריך הבחינה:	15.4.2024
שם המורה	ד"ר מני אדלר יאיר ועקנין
שם הקורס:	תכנות מערכות
מספר הקורס:	202-1-2031
שנה:	תשפ"ד
סמסטר:	א
מועד:	ב
משך הבחינה:	שעתיים
חומר עזר:	אין

רשמו תשובותיכם בגיליון התשובות בלבד, תשובות מחוץ לגיליון לא יבדקו.
ניתן לענות 'לא יודע' על סעיף שלם (בלבד) ולקבל 20% מהניקוד עבורו

בהצלחה!

שאלה 1: זיכרון (35 נקודות)

נתונה המחלקה BinaryTree ב-C++:

```
template <typename T>
class BinaryTree {

private:
    T data;

protected:
    BinaryTree<T>* left;
    BinaryTree<T>* right;

    virtual void clean() {
        if (left != nullptr) {
            delete left;
            left = nullptr;
        }
        if (right != nullptr) {
            delete right;
            right = nullptr;
        }
    }
```

```
    virtual void copy(const BinaryTree& other) {
        if (other.left == nullptr)
            left = nullptr;
        else
            left = new BinaryTree<T>(*(other.left));
        if (other.right == nullptr)
            right = nullptr;
        else
            right = new BinaryTree<T>(*(other.right));
    }

public:

    BinaryTree(T d) :  data(d), left(nullptr), right(nullptr) {}
    BinaryTree(T d, BinaryTree<T>* l, BinaryTree<T>* r) :
       data(d), left(l), right(r) {}

    BinaryTree(const BinaryTree<T>& other) : data(other.data) {
        copy(other);
    }

    virtual ~BinaryTree() { clean(); }

    BinaryTree<T>& operator=(const BinaryTree<T>& other) {
        if (this != &other) {
            data = other.data;
            clean();
            copy(other);
        }
        return *this;
    }

    BinaryTree(BinaryTree<T>&& other) :
        data(other.data), left(other.left), right(other.right) {
        other.left = nullptr;
        other.right = nullptr;
```

```
    }

    BinaryTree<T>& operator=(BinaryTree<T>&& other) {
        if (this != &other) {
            data = other.data;
            clean();
            left = other.left;
            right = other.right;
            other.left = nullptr;
            other.right = nullptr;
        }
        return *this;
    }

    T getData() const { return data; }
    BinaryTree<T>* getLeft() const { return left; }
    BinaryTree<T>* getRight() const { return right; }

    void insert(T d) {
        if (data == d)
            return;
        if (data > d) {
            if (left == nullptr)
                left = new BinaryTree<T>(d);
            else
                left->insert(d);
        } else {
            if (right == nullptr)
                right = new BinaryTree<T>(d);
            else
                right->insert(d);
        }
    }
};
```

```
template <typename T>
class Func {
public:
    virtual void apply(T& item) const = 0;
};
```

```
template <typename T>
class Traversable {
public:
    virtual void traverse(const Func<T>& f)=0;
};
```

המתודה traverse עוברת על הנתונים במבנה, על פי המדיניות המוגדרת במימוש, ומפעילה על כל אחד מהם את ה'פונקציה' f (= מבצעת על כל נתון f.apply)

א. כתבו את המחלקה InorderBinaryTree, המרחיבה את המחלקה BinaryTree על ידי מימוש ה'ממשק' Traversable.

- מספיק לממש בתבנית Rule of 3
- אין לכתוב מחדש פונקציות אם זה לא נדרש
- אין לשנות את המחלקה BinaryTree

<u>תזכורת</u>: קיימות שלוש דרכים לעבור על עץ בינארי:

- Preorder: קודם הנתון שבקודקוד, אחר כך הנתונים בתת העץ השמאלי, אחר כך הנתונים בתת העץ הימני
- Inorder: קודם הנתונים בתת העץ השמאלי, אחר כך הנתון שבקודקוד, אחר כך הנתונים בתת העץ הימני
- Postorder: קודם הנתונים בתת העץ השמאלי, אחר כך הנתונים בתת העץ הימני, אחר כך הנתון שבקודקוד.

[15 נקודות]

ב. אחת הסטודנטיות בקורס הציעה לשנות את הגדרת ה'ממשקים' באופן הבא:

```
template <typename T>
class Func {
public:
    virtual void apply(const T& item) const = 0;
};
```

```
template <typename T>
class Traversable {
public:
    virtual void traverse(const Func<T>& f) const = 0;
};
```

ציינו מה ההשלכה של שינוי זה.

[5 נקודות]

ג. אחד הסטודנטים בקורס הציע לשנות את הגדרת ה'ממשקים' באופן הבא:

```
template <typename T>
class Func {
public:
    virtual void apply(T& item) const = 0;
};
```

```
template <typename T>
class Traversable {
public:
    virtual void traverse(const Func<T>& f) const =0;
};
```

האם התוכנית תתקמפל? האם הייתם מקבלים את הצעתו? נמקו בקצרה.

[5 נקודות]

ד. נתונה המחלקה Printer והפונקציה g:

```
template <typename T>
class Print : public Func<T> {
    virtual void apply(T& item) const {
        cout << item;
    }
};

void g(Traversable<BinaryTree<int>>& t) {
    Print<BinaryTree<int>> printer;
    t.traverse(printer);
}
```

ציירו את תמונת הזיכרון בסיום קטע הקוד הבא:

```
{
    Traversable<BinaryTree<int>>* t1 = new InorderBinaryTree<BinaryTree<int>>(
        BinaryTree<int>(2),
        new InorderBinaryTree<BinaryTree<int>>(BinaryTree<int>(1)),
        new InorderBinaryTree<BinaryTree<int>>(BinaryTree<int>(3)));
    InorderBinaryTree<int> t2(2,
        new InorderBinaryTree<int>(1),
        new InorderBinaryTree<int>(3));
    InorderBinaryTree<int>& t3 = t2;
    g(*t1);
}
//@@ Draw the memory here
```

[10נקודות]

שאלה 2: מקביליות (30 נקודות)

המחלקה Barrier ('בָּרִיר') מאפשרת תיאום בין ת'רדים, כך שמשימותיהם יחולקו לשלבים משותפים שונים.
ת'רדים המשתמשים במחלקה זו יכולים להמתין בסיום כל שלב של משימתם עד ששאר הת'רדים יסיימו שלב זה.
לאחר שכל הת'רדים סיימו שלב זה, המחלקה תעביר אותם לשלב הבא של משימתם.

במחלקה ארבע מתודות:

public int await() throws InterruptedException
הת'רד המבצע את המתודה מציין כי הוא סיים את השלב הנוכחי של משימתו.
אם יש ת'רדים שלא סיימו שלב זה של משימתם, הת'רד המבצע עובר למצב blocked עד שהם יסיימו.
אם כל הת'רדים סיימו את השלב הזה של משימתם, כל הת'רדים ממשיכים לשלב הבא במשימתם.
המתודה מחזירה את מיקומו של הת'רד ברשימת הת'רדים שסיימו כבר שלב זה (אם הוא סיים ראשון חוזר 1, אם הוא סיים אחרון אחרון חוזר מספר כמספר הת'רדים)

public int getNumberWaiting()
המתודה מחזירה את מספר הת'רדים הממתינים כרגע (לאחר שסיימו לבצע את השלב הנוכחי של משימתם)

public int getParties()
המתודה מחזירה את מספר הת'רדים שלא סיימו עדיין את השלב הנוכחי.

public void reset()
החזרת ה-Barrier למצבו ההתחלתי, תוך החזרת הת'רדים הממתינים למצב ready.

א. השלימו את מימוש המחלקה בגיליון התשובות

```
public class Barrier {
    private final int threadsNum; // The number of threads in the system
    ...

    Barrier(int threadsNum) {
       this.threadsNum = threadsNum;
       ...
    }

    ...

}
```

[15 נקודות]

נתונה אחת הגרסאות של תבנית הפילוסופים-הרעבים שנלמדה בכיתה:

```
class Philosopher implements Runnable {

      private Fork left, right;

      Philosopher(Fork left, Fork right) {
         this.left = left; this.right = right;
      }

      public void run() {
         while (!Thread.currentThread().isInterrupted()) {
             think();
             if (System.identityHashCode(left) <
                  System.identityHashCode(right)) {
                 synchronized(left) {
                   synchronized(right) {
                         eat();
                   }
                 }
             }
             else {
                synchronized (right) {
                   synchronized  (left) {
                         eat();
                   }
                }
            }
       }
    }

    public static void main(String [] args) {

        Fork[] forks = new Fork[5];
        Philosopher[] philosophers = new Philosopher[5];
        for (int i=0; i<5; i++)
            forks[i] = new Fork();
        for (int i=0; i<5; i++)
            philosophers[i] = new Philosopher(
                                    forks[i % 5],forks[(i + 1) % 5]);
        for (int i=0; i<5; i++)
            new Thread(philosophers[i]).start();
    }

}
```

ב. השתמשו במחלקת ה-Barrier מסעיף א (ניתן לענות על סעיף זה גם אם לא מימשתם אותה) כך שלא ייווצר מצב שבו פילוסוף אוכל כאשר פילוסוף אחר חושב (זה לא מנומס…)
יש לכתוב קוד אך אין צורך להעתיק את כל הקוד הקיים - ציינו את המקומות בהם יש שינוי בקוד, ומה עדכנתם/הוספתם.
[5 נקודות]

ג. ציינו עבור כל אחד מהתרחישים הבאים האם הוא עשוי להתרחש בקוד המקורי, והאם הוא עשוי להתרחש לאחר עדכון הקוד בסעיף ב (ניתן לענות על סעיף זה גם אם לא עדכנתם את הקוד בפועל) ונמקו <u>בקצרה</u>: deadlock, livelock, starvation.

[10 נקודות]

שאלה 3: תקשורת ובסיסי נתונים (35 נקודות)

בנספח (בסוף השאלון), מופיע לנוחיותכם קוד הריאקטור כפי שנלמד בכיתה.

א. במסגרת ייעול המחסן הלוגיסטי עבור האזרחים המפונים, הוחלט להקים בסיס נתונים שירכז את כל המידע הנדרש לפעילות זו:

- מתנדב: מ.ז., שם, גיל, האם יש רשיון נהיגה בתוקף?
- משפחה מפונה: שם משפחה, כתובת זמנית (כמחרוזת), מספר נפשות (נתון כי אין משפחות בעלות שם זהה באותה כתובת זמנית, אין 'מספר מזהה למשפחה')
- מנת מזון: סוג מנה, כמות מנות זמינות כרגע
- רכב: מספר רכב, סוג רכב, תאריכים שבהם הרכב זמין לשימוש
- הזמנה: תאריך, סוג מנה, משפחה, מתנדב אחראי.

הגדירו מודל רלציוני (טבלאות, מפתחות) עבור המידע הנדרש למחסן הלוגיסטי.
אין לכתוב קוד SQL - ציירו תרשים כפי שנעשה בכיתה.
[10 נקודות]

נתונה המחלקה **DB** ב-**Java** המאפשרת ביצוע שאילתות על בסיס נתונים רלציוני.
אופן מימוש המחלקה אינו רלבנטי לשאלה זו. נתון כי היא Thread-Safe ושיש לה בנאי ריק.
המתודה המרכזית במחלקה היא **executeQuery**:

```
public List<String> executeQuery(String query) throws SQLException
```

המתודה מקבלת שאילתא, מריצה אותה על בסיס הנתונים, ומחזירה את מערך הרשומות שהשאילתא החזירה.
אם יש שגיאה בהרצת השאילתא, נזרקת חריגה מסוג **SQLException** (מימוש מחלקה זו אינו רלבנטי לשאלה)

ב. השלימו את התוכנית **DBServer** המריצה שרת מסוג Reactor עם עשרה ת'רדי עבודה, המקבל שאילתא מלקוח כמחרוזת, מריץ אותה על בסיס הנתונים, ומחזיר תשובה:
- אם השאילה החזירה רשומות: את הרשומה הראשונה.
- אם השאילה לא החזירה רשומות: את ההודעה "No Result"
- אם היתה שגיאה בהרצת השאילתא: את הודעת השגיאה

הערה: אתחול האובייקט **DB** הוא כבד - אין להגדיר מופעים מיותרים שלו.

```
...
class DBServer {
    public static void main(String[] args) {
        int port = IntegerParseInt(args[0]);
        ...
    }
}
```

[10 נקודות]

אחד הסטודנטים בקורס החליט לייעל את עבודת יחידת התקשורת בריאקטור, על ידי עדכון המתודות **handleAccept, handleReadWrite** במחלקה **Reactor**, ואת המתודה **continueWrite** במחלקה **NonBlockingConnectionHandler**, באופן הבא:

```
class Reactor {

  ActorThreadPool<Object> communicationThreads = new ActorThreadPool<Object>(3);
  ...

 private void handleAccept(ServrSocketChannel serverChan,
                           Selector selector) throws IOException {
     communicationThreads.submit(this, () -> {
        SocketChannel clientChan = serverChan.accept();

        clientChan.configureBlocking(false);
        final NonBlockingConnectionHandler<T> handler = new
             NonBlockingConnectionHandler<>(
                readerFactory.get(),
                protocolFactory.get(),
                clientChan,
                this);
        clientChan.register(selector, SelectionKey.OP_READ, handler);
     });
   }

 private void handleReadWrite(SelectionKey key) {
        @SuppressWarnings("unchecked")
        NonBlockingConnectionHandler<T> handler =
                    (NonBlockingConnectionHandler<T>) key.attachment();
        if (key.isReadable()) {
            communicationThreads.submit(handler, () -> {
               Runnable task = handler.continueRead();
               if (task != null)
                   pool.submit(handler, task);
               else if (!key.isValid())
                   return;
            });
        }
        if (key.isWritable()) {
           communicationThreads.submit(handler, () -> {
               handler.continueWrite();
           });
        }
    }
    ...
}
```

```
class NonBlockingConnectionHandler<T>  implements ConnectionHandler<T> {

...

public Runnable continueRead() {

    ...
    return () -> {
        try {
          while (buf.hasRemaining()) {
             T nextMessage = encdec.decodeNextByte(buf.get());
             if (nextMessage != null) {
                T response = protocol.process(nextMessage);
                if (response != null) {
                  synchronized (writeQueue) {
                     writeQueue.add(ByteBuffer.wrap(
                                    encdec.encode(response)));
                     reactor.updateInterestedOps(chan, SelectionKey.OP_READ |
                                                       SelectionKey.OP_WRITE);
                  }
                }
             }
          }
        } finally {
          releaseBuffer(buf);
        }
      };

      ...
  }

  public void continueWrite() {
    while (!writeQueue.isEmpty()) {
      try {
        ByteBuffer top = writeQueue.peek();
        chan.write(top);
        if (top.hasRemaining()) {
          return;
        } else {
          writeQueue.remove();
        }
      } catch (IOException ex) {
        ex.printStackTrace();
        close();
      }
    }
```

```
    synchronized (writeQueue) {
      if (writeQueue.isEmpty()) {
        if (protocol.shouldTerminate()) close();
        else reactor.updateInterestedOps(chan, SelectionKey.OP_READ);
      }
    }
  }

...

}
```

בתיעוד של Java מצוין כי:

- המחלקה SocketChannel היא Thread Safe
- המחלקה ByteBuffer אינה Thread Safe
- המחלקה ServerSocketChannel אינה Thread Safe
- המחלקות Selector ו-SelectionKey הן ThreadSafe

ג. ציינו מה הבעיה בהצעה של הסטודנט, ותקנו את הקוד כך שהיא תיפתר (התיקון צריך לשמר את ההצעה של הסטודנט בדבר 3 ת'רדים נוספים לניהול התקשורת - אל תורידו ת'רדים מהמערכת)
יש לכתוב קוד אך אין צורך להעתיק את כל הקוד הקיים - ציינו את המקומות בהם יש שינוי בקוד, ומה עדכנתם/הוספתם.

[15 נקודות]

נספח: קוד הריאקטור, כפי שנלמד בכיתה

```
public class Reactor<T> implements Server<T> {

    private final int port;
    private final Supplier<MessagingProtocol<T>> protocolFactory;
    private final Supplier<MessageEncoderDecoder<T>> readerFactory;
    private final ActorThreadPool<NonBlockingConnectionHandler> pool;
    private Selector selector;

    private Thread selectorThread;
    private final ConcurrentLinkedQueue<Runnable> selectorTasks = new
                                            ConcurrentLinkedQueue<>();

    public Reactor(
            int numThreads,
            int port,
            Supplier<MessagingProtocol<T>> protocolFactory,
            Supplier<MessageEncoderDecoder<T>> readerFactory) {

        this.pool = new ActorThreadPool<>(numThreads);
        this.port = port;
        this.protocolFactory = protocolFactory;
        this.readerFactory = readerFactory;
    }

    public void serve() {
        selectorThread = Thread.currentThread();

        try (   Selector selector = Selector.open();
                ServerSocketChannel serverSock =
                                    ServerSocketChannel.open()) {

            this.selector = selector; //just to be able to close

            serverSock.bind(new InetSocketAddress(port));
            serverSock.configureBlocking(false);
            serverSock.register(selector, SelectionKey.OP_ACCEPT);

            while (!Thread.currentThread().isInterrupted()) {

                selector.select();
                runSelectionThreadTasks();

                for (SelectionKey key : selector.selectedKeys()) {
```

```
                    if (!key.isValid()) {
                        continue;
                    } else if (key.isAcceptable()) {
                        handleAccept(serverSock, selector);
                    } else {
                        handleReadWrite(key);
                    }
                }

                selector.selectedKeys().clear();

            }

        } catch (ClosedSelectorException ex) {
            //do nothing - server was requested to be closed
        } catch (IOException ex) {
            //this is an error
            ex.printStackTrace();
        }

        System.out.println("server closed!!!");
        pool.shutdown();
    }

    void updateInterestedOps(SocketChannel chan, int ops) {
        final SelectionKey key = chan.keyFor(selector);
        if (Thread.currentThread() == selectorThread) {
            key.interestOps(ops);
        } else {
            selectorTasks.add(() -> {
                if(key.isValid())
                    key.interestOps(ops);
            });
            selector.wakeup();
        }
    }



private void handleAccept(ServerSocketChannel serverChan,
                          Selector selector) throws IOException {
        SocketChannel clientChan = serverChan.accept();

        clientChan.configureBlocking(false);
        final NonBlockingConnectionHandler<T> handler = new
```

```
            NonBlockingConnectionHandler<>(
               readerFactory.get(),
               protocolFactory.get(),
               clientChan,
               this);

        clientChan.register(selector, SelectionKey.OP_READ, handler);
    }

  private void handleReadWrite(SelectionKey key) {
        @SuppressWarnings("unchecked")
        NonBlockingConnectionHandler<T> handler =
                    (NonBlockingConnectionHandler<T>) key.attachment();
        if (key.isReadable()) {
            Runnable task = handler.continueRead();
            if (task != null) {
                pool.submit(handler, task);
            } else if (!key.isValid()) {
                return;
            }
        }
        if (key.isWritable()) {
            handler.continueWrite();
        }
    }

   private void runSelectionThreadTasks() {
     while (!selectorTasks.isEmpty()) {
       selectorTasks.remove().run();
     }
   }

   public void close() throws IOException {
     selector.close();
   }
 }

public class NonBlockingConnectionHandler<T> implements
                                            ConnectionHandler<T> {
  private static final int BUFFER_ALLOCATION_SIZE = 1 << 13; //8k
  private static final ConcurrentLinkedQueue<ByteBuffer> BUFFER_POOL =
                                        new ConcurrentLinkedQueue<>();

  private final MessagingProtocol<T> protocol;
  private final MessageEncoderDecoder<T> encdec;
  private final Queue<ByteBuffer> writeQueue = new ConcurrentLinkedQueue<>();
```

```
private final SocketChannel chan;
private final Reactor reactor;

public NonBlockingConnectionHandler(
    MessageEncoderDecoder<T> reader,
    MessagingProtocol<T> protocol,
    SocketChannel chan,
    Reactor reactor) {
      this.chan = chan;
      this.encdec = reader;
      this.protocol = protocol;
      this.reactor = reactor;
}

public Runnable continueRead() {
  ByteBuffer buf = leaseBuffer();
  boolean success = false;
  try {
    success = chan.read(buf) != -1;
  } catch (IOException ex) {
    ex.printStackTrace();
  }
  if (success) {
    buf.flip();
    return () -> {
      try {
        while (buf.hasRemaining()) {
          T nextMessage = encdec.decodeNextByte(buf.get());
          if (nextMessage != null) {
            T response = protocol.process(nextMessage);
            if (response != null) {
              writeQueue.add(ByteBuffer.wrap(
                                encdec.encode(response)));
              reactor.updateInterestedOps(chan, SelectionKey.OP_READ
                                              | SelectionKey.OP_WRITE);
            }
          }
        }
      } finally {
        releaseBuffer(buf);
      }
    };
  } else {
    releaseBuffer(buf);
    close();
    return null;
  }
```

```
    }

  public void close() {
    try {
      chan.close();
    } catch (IOException ex) {
        ex.printStackTrace();
    }
  }

  public void continueWrite() {
    while (!writeQueue.isEmpty()) {
      try {
        ByteBuffer top = writeQueue.peek();
        chan.write(top);
        if (top.hasRemaining()) {
          return;
        } else {
          writeQueue.remove();
        }
      } catch (IOException ex) {
        ex.printStackTrace();
        close();
      }
    }
    if (writeQueue.isEmpty()) {
      if (protocol.shouldTerminate()) close();
        else reactor.updateInterestedOps(chan, SelectionKey.OP_READ);
    }
  }

  private static ByteBuffer leaseBuffer() {
    ByteBuffer buff = BUFFER_POOL.poll();
    if (buff == null) {
      return ByteBuffer.allocateDirect(BUFFER_ALLOCATION_SIZE);
    }
    buff.clear();
    return buff;
  }

  private static void releaseBuffer(ByteBuffer buff) {
    BUFFER_POOL.add(buff);
  }
}
```

```
public class ActorThreadPool<T> {

  private final Map<T, Queue<Runnable>> actors;
  private final ReadWriteLock actorsReadWriteLock;
  private final Set<T> playingNow;
  private final ExecutorService threads;

  public ActorThreadPool(int threads) {
      this.threads = Executors.newFixedThreadPool(threads);
      actors = new WeakHashMap<>();
      playingNow = ConcurrentHashMap.newKeySet();
      actorsReadWriteLock = new ReentrantReadWriteLock();
  }

  public void shutdown() {
    threads.shutdownNow();
  }

  public void submit(T actor, Runnable r) {
    synchronized (actor) {
      if (!playingNow.contains(actor)) {
        playingNow.add(actor);
        execute(r, actor);
      } else {
        pendingRunnablesOf(actor).add(r);
      }
    }
  }

  private Queue<Runnable> pendingRunnablesOf(T actors) {
    actorsReadWriteLock.readLock().lock();
    Queue<Runnable> pendingRunnables = actors.get(actors);
    actorsReadWriteLock.readLock().unlock();

    if (pendingRunnables == null) {
      actorsReadWriteLock.writeLock().lock();
      actors.put(actor, pendingRunnables = new LinkedList<>());
      actorsReadWriteLock.writeLock().unlock();
    }
    return pendingRunnables;
  }

  private void execute(Runnable r, T actor) {
    threads.submit(() -> {
      try {
        r.run();
      } finally {
```

```
          complete(actor);
        }
      });
    }

    private void complete(T actor) {
      synchronized (actor) {
        Queue<Runnable> pending = pendingRunnablesOf(actor);
        if (pending.isEmpty()) {
          playingNow.remove(actor);
        } else {
          execute(pending.poll(), actor);
        }
      }
    }
  }
```

```
public interface MessagingProtocol<T> {
    public T process(T msg);
    public boolean shouldTerminate();
}

public class EchoProtocol implements MessagingProtocol<String> {

    private boolean shouldTerminate = false;

    @Override
    public String process(String msg) {
        shouldTerminate = "bye".equals(msg);
        System.out.println("[" + LocalDateTime.now() + "]: " + msg);
        return createEcho(msg);
    }

    private String createEcho(String message) {
        String echoPart = message.substring(
                Math.max(message.length() - 2, 0), message.length());
        return message + " .. " + echoPart + " .. " + echoPart + " ..";
    }

    @Override
    public boolean shouldTerminate() {
        return shouldTerminate;
    }
}
```

```java
public interface MessageEncoderDecoder<T> {
  T decodeNextByte(byte nextByte);
  byte[] encode(T message);
}

public class LineMessageEncoderDecoder implements MessageEncoderDecoder<String> {

    private byte[] bytes = new byte[1 << 10]; //start with 1k
    private int len = 0;

    @Override
    public String decodeNextByte(byte nextByte) {
         if (nextByte == '\n') {
            return popString();
        }

        pushByte(nextByte);
        return null; //not a line yet
    }

    @Override
    public byte[] encode(String message) {
        return (message + "\n").getBytes(); //uses utf8 by default
    }

    private void pushByte(byte nextByte) {
        if (len >= bytes.length) {
            bytes = Arrays.copyOf(bytes, len * 2);
        }

        bytes[len++] = nextByte;
    }

    private String popString() {
        String result = new String(bytes, 0, len, StandardCharsets.UTF_8);
        len = 0;
        return result;
    }
}
```